ROADRESQ

Master Implementation Plan

A phased, dependency-driven plan for building the production platform

Document Type: Product + Engineering Implementation Blueprint
Version: 1.0 **Status:** Master Build Plan
Date: 04 September 2026

Core principle: Build RoadResQ in vertical slices. Establish the foundation first, then make one complete customer-to-provider journey work end-to-end before expanding the platform.

Important: This plan is an implementation blueprint, not a promise of fixed delivery dates. Actual duration depends on team size, experience, scope changes, vendor availability, testing and production constraints.

1. Implementation Objective

The objective is to transform the RoadResQ product and technical documentation into a working, testable, deployable roadside-assistance platform. The implementation must prioritize a reliable core journey over building dozens of disconnected features.

Primary MVP outcome: A customer can create a roadside assistance request, provide a vehicle and location, receive a suitable provider, have the provider accept the job, track progress, complete the service, approve an estimate when required, pay, and submit a review. A provider can onboard, be verified, receive requests, accept jobs, update status, complete service, and receive settlement records. Admin can operate the marketplace safely.

2. Build Philosophy

- **Foundation before features:** Authentication, database, authorization, environments and observability must exist before high-value workflows.
- **Vertical slice over isolated modules:** Build a complete thin version of the main journey before polishing every individual screen.
- **Backend contract first:** Define stable API contracts and domain rules before frontend integration.
- **State machines over ad-hoc flags:** Booking, verification, payment and job states must have explicit legal transitions.
- **Safety by default:** Location, payment, provider verification and privileged operations require strong controls from the beginning.
- **Production thinking from day one:** Logging, migrations, backups, testing and deployment are part of implementation, not post-launch cleanup.
- **Measure before optimizing:** Instrument important journeys before making performance or product decisions.

3. Recommended Technical Baseline

Layer	Recommended Implementation
Customer Web	Next.js + React + TypeScript + Tailwind
Customer Mobile	React Native / Expo
Provider Interface	Responsive web initially or React Native if mobile-first operations are required
Admin	Next.js / React with strict RBAC
Backend	FastAPI + Python, modular monolith
Database	PostgreSQL + PostGIS
Cache / Ephemeral State	Redis
Realtime	WebSockets with safe polling fallback
Background Jobs	Worker/queue architecture
Storage	S3-compatible object storage
Maps	Google Maps or Mapbox, selected and contracted before integration
Payments	India-capable payment gateway supporting required methods and webhooks
Notifications	Push + SMS/OTP + email as required
Deployment	Docker-based environments with CI/CD
Observability	Metrics + structured logs + traces + alerting

4. Master Phase Overview

Phase	Name	Primary Outcome	Why This Phase Exists
0	Project Foundation	Repository, environments, standards, architecture baseline	Prevents chaotic development and environment drift
1	Data + Backend Foundation	Database, auth, RBAC, core domain APIs	Creates the reliable system of record
2	Customer Foundation	Customer account, vehicles, service request UX	Creates the demand side of the marketplace
3	Provider Foundation	Provider onboarding, verification, availability, profile	Creates the supply side needed for matching
4	Core Dispatch	Matching, accept/reject, booking state machine	Creates the actual marketplace transaction
5	Live Assistance	Tracking, arrival, job execution, inspection	Turns a booking into a real roadside service
6	Billing + Trust	Estimates, invoices, payments, reviews, support	Makes the service commercially complete
7	Admin + Operations	Verification, monitoring, disputes, controls	Makes the platform operable at scale
8	Hardening + Production	Security, QA, performance, DR, deployment	Makes the system safe and reliable
9	Pilot + Launch	Controlled real-world rollout	Validates product assumptions before broad launch
10	Post-MVP Expansion	Scheduled service, garage OS, intelligence, ecosystem	Expands after the core marketplace is proven

5. Phase 0 — Project Foundation

Goal

Create a clean engineering foundation so every later phase uses the same architecture, environment, conventions and delivery process.

Build

- Create repository structure and protected main branch.
- Define frontend, backend, mobile/provider/admin application boundaries.
- Create local, development, staging and production configuration strategy.
- Set environment-variable and secret-management conventions.
- Set up Docker for reproducible local/deployment environments.
- Set up PostgreSQL/PostGIS and Redis locally.
- Establish formatting, linting, type checking and commit/PR conventions.
- Create CI pipeline for build, lint, type check and tests.
- Create initial observability hooks and correlation ID standard.
- Create database migration system and seed strategy.

Exit Gate

A new developer can clone the project, configure the approved environment, run the stack, execute migrations, run tests, and start the application without undocumented manual steps.

6. Phase 1 — Data + Backend Foundation

Goal

Build the core system of record and secure API foundation before implementing complex customer/provider workflows.

Build Order

- users and authentication

- roles and permissions
- vehicles
- providers and provider profiles
- services and provider capabilities
- bookings
- booking locations
- booking status history
- provider availability
- audit logs
- notifications
- support cases
- payment/invoice foundations

Engineering Work

- Implement FastAPI application structure by domain module.
- Implement database models, migrations, indexes and constraints.
- Implement Pydantic request/response schemas.
- Implement authentication and token/session lifecycle.
- Implement RBAC and object-level authorization.
- Implement consistent error and pagination contracts.
- Implement API versioning and OpenAPI documentation.
- Implement audit events for sensitive actions.
- Implement unit and integration test foundations.

Exit Gate

Secure authentication works, authorized users can access only permitted objects, migrations are reproducible, core entities persist correctly, and automated API/database tests pass.

7. Phase 2 — Customer Foundation

Goal

Enable a customer to create a complete, valid assistance request.

Build

- Customer registration/login.
- Profile and contact management.
- Vehicle CRUD and vehicle selection.
- Service/problem selection.
- Current location acquisition with permission handling.
- Manual location fallback.
- Problem description.
- Photo/video upload where supported.

- Emergency-service disclaimer.
- Request review and submission.
- Request status screen.
- Basic request history.

Why Now

Without a reliable customer request, provider and dispatch development has nothing meaningful to consume. This phase establishes the demand-side contract.

Exit Gate

A test customer can submit a valid request from a supported device/browser, the backend stores it correctly, and the request appears in an operational/admin view.

8. Phase 3 — Provider Foundation

Goal

Create trustworthy and operationally usable service supply.

Build

- Provider registration.
- Business/profile information.
- Identity/address/document upload.
- Service capability configuration.
- Supported vehicle/service types.
- Service radius.
- Working hours and availability.
- Online/offline state.
- Verification workflow.
- Provider dashboard.
- Incoming-request foundation.

Why Now

RoadResQ cannot validate its marketplace model until verified providers can actually participate. Provider verification must exist before public dispatch.

Exit Gate

A provider can register, submit verification evidence, be approved by an admin/test operator, configure availability and appear as eligible for matching.

9. Phase 4 — Core Dispatch & Booking

Goal

Make the central marketplace transaction work reliably.

Build

- Eligible-provider query.

- Service compatibility filter.
- Vehicle compatibility filter.
- Availability filter.
- Distance/ETA calculation.
- Provider ranking.
- Dispatch request.
- Provider accept/reject.
- Timeout and retry.
- Assignment locking to prevent double assignment.
- Customer/provider booking views.
- Explicit booking state machine.
- Cancellation handling.

Core State Machine

REQUESTED → MATCHING → OFFERED → ACCEPTED → EN_ROUTE → ARRIVED → INSPECTION → ESTIMATE_PENDING → APPROVED → IN_PROGRESS → COMPLETED → INVOICED → PAID → REVIEWED

Alternative transitions such as REJECTED, CANCELLED, EXPIRED, FAILED, DISPUTED and SUSPENDED must be defined and implemented rather than represented through informal flags.

Why This Is the MVP Center

This phase proves the actual marketplace: demand meets supply and a real job is created. Everything before it supports this transaction; much of what follows completes and hardens it.

Exit Gate

In staging, a real end-to-end test can create a request, match one eligible provider, accept exactly once, and maintain a consistent booking state under retries and concurrent actions.

10. Phase 5 — Live Assistance & Job Execution

Goal

Turn an accepted booking into an operational roadside-assistance job.

Build

- Provider live location updates.
- Customer map/tracking view.
- WebSocket events.
- Polling fallback.
- ETA display.
- Provider arrival status.
- Arrival OTP.
- Job card.
- Inspection.
- Inspection items/evidence.
- Estimate creation.

- Customer estimate approval.
- Additional-work approval.
- Service completion.
- Completion evidence.

Why Now

Dispatch alone does not deliver value. This phase connects the digital booking to the physical service and creates the evidence required for billing and disputes.

Exit Gate

A provider can travel to the customer, update arrival, complete inspection/estimate, perform the job, submit completion evidence and close the service safely.

11. Phase 6 — Billing, Payments & Trust

Goal

Make completed jobs commercially complete and trustworthy.

Build

- Estimate and approval enforcement.
- Invoice generation.
- Payment intent/order creation.
- UPI/card/approved payment methods as selected.
- Webhook processing.
- Idempotency and duplicate protection.
- Payment pending/failed/success states.
- Refund workflow.
- Provider earnings.
- Payout records.
- Settlement and reconciliation.
- Customer receipt.
- Provider statement.
- Ratings and reviews.
- Basic dispute initiation.

Why Now

Payments should be integrated after the service workflow is stable enough to define what is actually being charged. Financial state must remain independent and auditable.

Exit Gate

A completed job can generate an invoice, accept a payment, process delayed/duplicate webhook events safely, reconcile the financial state and record provider settlement.

12. Phase 7 — Admin & Operational Control

Goal

Give the operations team the tools needed to run the marketplace safely without direct database manipulation.

Build

- Admin authentication and strict RBAC.
- Customer management.
- Provider verification queue.
- Provider suspension/reinstatement.
- Booking monitoring.
- Dispatch visibility.
- Payment/reconciliation view.
- Support/dispute management.
- Fraud/risk review controls.
- Audit log viewer.
- Operational dashboards.
- Feature flags / controlled kill switches.
- Manual recovery tools with audit trails.

Why Now

Operational controls should exist before production exposure. Otherwise support staff are forced into unsafe manual database operations during incidents.

Exit Gate

Operations can verify providers, monitor active bookings, investigate disputes, handle common failures and audit privileged actions without bypassing system controls.

13. Phase 8 — Production Hardening

Goal

Move from 'works in staging' to 'safe to operate in production.'

Security

- Threat-model verification.
- RBAC/object-level authorization testing.
- Rate limiting and abuse controls.
- Secure upload validation.
- Secret rotation verification.
- Dependency and container scanning.
- Admin access review.
- Security logging and alerting.
- Privacy-data exposure review.

Quality

- Unit tests for domain rules.
- API/contract tests.

- Database integration tests.
- Booking state-machine tests.
- Payment idempotency tests.
- Realtime disconnect/reconnect tests.
- Provider cancellation and timeout tests.
- Accessibility checks.
- Cross-browser/device testing.
- Regression suite.

Reliability

- Load/performance testing.
- Database query/index review.
- Queue failure testing.
- Redis failure/degradation testing.
- External dependency failure testing.
- Backup restore test.
- Disaster recovery drill.
- Incident-response tabletop.

Exit Gate

All production readiness gates pass, critical defects are closed or formally accepted, recovery has been tested, monitoring is active, and the team can operate the system using documented runbooks.

14. Phase 9 — Controlled Pilot

Goal

Validate RoadResQ with a small, controlled real-world network before a broad public launch.

Pilot Design

- Select a limited geographic area.
- Onboard a controlled number of verified providers.
- Invite a limited customer group.
- Use real support coverage.
- Monitor every critical booking journey.
- Track failed matches, cancellations, ETA accuracy, payment issues and complaints.
- Review provider quality daily during the pilot.
- Keep manual operational fallback available.

Pilot Exit Criteria

Metric / Gate	Pilot Standard
Critical safety issue	Zero unresolved critical safety process failures
Booking reliability	Core journey completes consistently in controlled tests and pilot traffic
Payment integrity	No unexplained financial discrepancies

Metric / Gate	Pilot Standard
Provider quality	No systemic unresolved abuse/quality pattern
Support	Critical cases have documented escalation and resolution paths
Availability	Core service meets internally defined SLO targets
Recovery	Incident and backup recovery procedures have been exercised

15. Phase 10 — Production Launch

Launch Sequence

- Freeze non-essential schema and feature changes.
- Confirm production backups and restore capability.
- Run final migration rehearsal.
- Confirm secrets and third-party credentials.
- Confirm payment/webhook configuration.
- Confirm maps/OTP/notification quotas and vendor support.
- Run production smoke tests.
- Enable monitoring and alerts.
- Enable support and incident-response coverage.
- Launch using staged traffic or feature flags.
- Monitor critical journeys continuously during the launch window.
- Roll back or disable features when predefined safety thresholds are exceeded.

16. Frontend Implementation Order

Order	Customer	Provider	Admin
1	Auth + onboarding	Auth + onboarding	Auth + RBAC
2	Vehicle management	Profile + verification	Provider verification
3	GET HELP NOW	Availability	Booking monitoring
4	Request creation	Incoming requests	User/provider management
5	Matching state	Accept/reject	Dispatch visibility
6	Live tracking	En-route/arrival	Operational support
7	Inspection/estimate approval	Inspection/estimate	Dispute management
8	Payment/invoice	Completion/earnings	Payments/reconciliation
9	History/reviews	History	Analytics/audit

17. Backend Implementation Order

- 1. Application bootstrap, configuration, health checks and database connection.
- 2. Users, authentication, roles and authorization.
- 3. Vehicles and service catalog.
- 4. Provider profiles, services, availability and verification.

- 5. Bookings, locations and state history.
- 6. Matching and dispatch transaction logic.
- 7. Realtime location and booking events.
- 8. Job cards, inspections and estimates.
- 9. Invoices, payments, refunds, payouts and reconciliation.
- 10. Notifications and background jobs.
- 11. Support, disputes and administrative controls.
- 12. Analytics, audit, observability and production hardening.

18. Database Migration Order

Sequence	Database Area	Reason
1	users / roles / auth	Foundation for ownership and authorization
2	vehicles	Required by customer requests
3	services	Required by matching
4	providers / provider documents	Supply-side foundation
5	provider services / availability	Eligibility and dispatch
6	bookings / booking locations	Core transaction
7	booking status history	Auditability and state reconstruction
8	provider location updates	Realtime tracking
9	job cards / inspections / estimates	Physical service workflow
10	invoices / payments / payouts	Financial lifecycle
11	reviews / notifications / messages	Trust and communication
12	support / audit / analytics	Operations and governance

19. Testing Strategy by Phase

Phase	Minimum Testing Before Moving Forward
0	Build/lint/type checks; environment reproducibility
1	Auth, RBAC, API contracts, DB constraints and migrations
2	Request validation, location permissions/fallbacks, uploads, UI states
3	Verification lifecycle, document access, provider eligibility
4	Concurrency, idempotency, state-machine transitions, dispatch timeouts
5	WebSocket reconnect, stale GPS, OTP, estimate approval, job completion
6	Payment webhooks, duplicates, refunds, reconciliation, invoice integrity
7	Admin authorization, auditability, support/dispute workflows
8	Full regression, load, security, accessibility, recovery and resilience
9	Pilot smoke/regression plus production monitoring validation

20. Dependency Rules

- Do not build live tracking before booking ownership and authorization are stable.

- Do not build payment settlement before invoice and job-completion semantics are stable.
- Do not expose provider dispatch before provider verification and suspension controls exist.
- Do not allow production admin actions without audit logging.
- Do not rely on frontend state to define financial or booking truth.
- Do not launch continuous location collection without corresponding privacy/access controls.
- Do not perform destructive DB fixes manually without an approved recovery/evidence procedure.
- Do not scale infrastructure before measuring actual bottlenecks.

21. Recommended Sprint Structure

Use 1–2 week engineering sprints. Each sprint should end with a demonstrable increment, automated tests and updated documentation.

Sprint Pattern	Activities
Day 1	Refine acceptance criteria, dependencies and technical design
Days 2–7	Implementation + unit/integration testing
Days 5–8	Frontend integration + edge cases
Days 8–9	QA/regression/security checks
Day 10	Demo, defect triage, documentation, release decision

For a small team, reduce parallel work. One complete workflow is more valuable than five half-built modules.

22. Team Ownership Model

Workstream	Primary Owner	Supporting
Product requirements	Product	Operations + Engineering
Backend/domain	Backend	Database + Security
Frontend/mobile	Frontend	Product + Design
Provider operations	Operations	Product + Support
Admin	Full-stack	Operations + Security
Database	Backend/DB	DevOps
Security	Security/Engineering	All teams
QA	QA/Engineering	Product + Operations
DevOps/SRE	DevOps	Backend + Security
Payments	Backend + Finance	Legal + Support

23. Definition of Done for Every Feature

- Acceptance criteria are met.
- UI states include loading, empty, success, error and relevant offline/permission states.
- API validation and authorization are implemented.
- Database constraints and migrations are included where needed.
- Unit/integration tests cover business-critical behavior.
- Audit/security implications are considered.

- Observability events/metrics are added where operationally relevant.
- Accessibility requirements are met.
- Documentation is updated.
- Staging deployment succeeds.
- Product owner/QA accepts the feature.

24. Critical MVP Scope — What NOT to Build First

- Do not start with a complex AI recommendation engine.
- Do not start with a full garage ERP/CRM.
- Do not start with nationwide provider onboarding.
- Do not build a microservices architecture before scale requires it.
- Do not build sophisticated surge pricing before baseline demand/supply data exists.
- Do not build advanced loyalty/subscription programs before the core service works.
- Do not build multiple payment providers unless redundancy is a launch requirement.
- Do not over-engineer analytics before the core business events are instrumented.

MVP rule: If a feature does not help a customer get assistance, help a provider deliver assistance, help RoadResQ operate the transaction, or make that transaction safe/reliable, it should usually wait.

25. First Complete Vertical Slice

The first major milestone should be a thin but real end-to-end slice:

- Customer logs in.
- Customer has one vehicle.
- Customer selects a roadside problem.
- Customer shares location.
- Customer submits GET HELP NOW.
- Backend finds one eligible verified provider.
- Provider receives request.
- Provider accepts.
- Customer sees accepted provider.
- Provider updates EN_ROUTE.
- Customer sees provider state/location.
- Provider marks ARRIVED.
- Provider completes a simple job record.
- Customer sees completion.
- A basic invoice/payment record is created.
- Customer submits a review.

This slice should be intentionally simple. Once it works reliably, each subsequent phase increases depth, safety, automation and coverage.

26. Release Gates

Gate	Must Be True
G1 Foundation	Builds, migrations, environments and CI are stable
G2 Security	Authentication, RBAC and object-level authorization verified
G3 Core Journey	Customer request → provider acceptance works
G4 Service	Tracking → arrival → job completion works
G5 Money	Invoice → payment → reconciliation works
G6 Operations	Admin/support can operate and recover common cases
G7 Reliability	Monitoring, backups, incident response and recovery tested
G8 Launch	Pilot criteria satisfied and production sign-off complete

27. Risk Management During Implementation

Risk	Prevention / Response
Scope explosion	Freeze MVP scope; maintain backlog for post-MVP
Architecture churn	Use ADRs and review major changes before implementation
Frontend/backend mismatch	Use API contracts and staging integration early
Unsafe manual fixes	Build admin recovery tools and audited operations
Payment inconsistency	Use idempotency, immutable financial events and reconciliation
Provider supply shortage	Pilot in a controlled area and measure provider availability
Poor GPS quality	Support manual location and stale-state UX
Third-party outage	Design fallbacks and vendor escalation paths
Security regression	Automated security checks + review of sensitive changes
Technical debt	Track explicitly and reserve capacity for remediation

28. What Happens After MVP

- **Phase A:** Improve reliability, provider density and customer conversion.
- **Phase B:** Scheduled service, garage discovery and service history expansion.
- **Phase C:** Better provider/garage operations and inventory/workshop capabilities.
- **Phase D:** Intelligent ETA, provider ranking, fraud/risk signals and demand forecasting.
- **Phase E:** Business partnerships, fleets, insurance/assistance integrations and broader automotive ecosystem.

Post-MVP expansion should be driven by measured operational bottlenecks and customer/provider demand rather than by documentation volume or feature count.

29. Master Implementation Checklist

- ■ Repository and environments created
- ■ CI/CD baseline working
- ■ Database and migrations working
- ■ Authentication + RBAC working
- ■ Customer + vehicle flow working
- ■ Provider onboarding + verification working

- ■ Assistance request working
- ■ Matching + dispatch working
- ■ Booking state machine enforced
- ■ Realtime tracking working with fallback
- ■ Job/inspection/estimate flow working
- ■ Invoice/payment/reconciliation working
- ■ Reviews/support working
- ■ Admin operations working
- ■ Security testing complete
- ■ QA/regression complete
- ■ Performance/load checks complete
- ■ Backup/restore tested
- ■ Incident response tested
- ■ Pilot completed
- ■ Production launch approved

30. Final Implementation Principle

RoadResQ should not be built as a collection of screens, APIs, or database tables. It should be built as a sequence of reliable business capabilities. Each phase must produce something that can be demonstrated, tested, observed and recovered. The critical path is:

FOUNDATION → DATA → AUTH → CUSTOMER REQUEST → VERIFIED PROVIDER → MATCHING → ACCEPTANCE → LIVE ASSISTANCE → JOB → PAYMENT → SUPPORT/OPERATIONS → HARDENING → PILOT → PRODUCTION.

If this sequence is followed, the project gains a working core early, limits architectural risk, creates measurable milestones, and gives the team a clear path from development to a production-ready RoadResQ platform.